

Software requirements and risk management

Dave, the project manager for the Chemical Tracking System at Contoso Pharmaceuticals, is meeting with his lead programmer, Helen, and the lead tester, Ramesh. All are excited about the new project, but they remember the problems they ran into on an earlier project called the Pharm Simulator.

"Remember how we didn't find out that the users hated the Simulator's user interface until beta testing?" Helen asked. "It took us four weeks to rebuild it and retest it. I sure don't want to go through that death march again."

"That was awful," Dave agreed. "It was also annoying that the users we talked to swore they needed a lot of features that no one has used so far. That drug interaction modeling feature took three times longer to code than we expected, and we wound up throwing it out. What a waste!"

Ramesh had a suggestion. "Maybe we should list these problems from the Simulator so we can try to avoid them on the Chemical Tracking System. I read an article on software risk management that said we should identify risks up front and figure out how to prevent them from hurting the project."

"I don't know about that," Dave protested. "We probably won't have those same problems again. If we write down things that could go wrong on the Chemical Tracking System, it'll look like I don't know how to run a software project. I don't want any negative thinkers on this project. We have to plan for success!"

As Dave's final comment suggests, software engineers and project managers are eternal optimists. We often expect our next project to run smoothly, despite the history of problems on earlier projects. The reality is that dozens of potential pitfalls can delay or derail a software project. Contrary to Dave's beliefs, software teams *must* identify and control their project risks, beginning with those related to requirements.

A *risk* is a condition that could cause some loss or otherwise threaten the success of a project. This condition hasn't actually caused a problem yet—and you'd like to keep it that way. These potential problems might have an adverse impact on the project's cost, schedule, or technical success; the product's quality; or the team's effectiveness. *Risk management* is the process of identifying, evaluating, and controlling risks before they harm your project. If something untoward has already happened on the project, it's an issue, not a risk. Deal with current problems and issues through your project's ongoing status tracking and corrective action processes.

Because no one can predict the future with certainty, risk management is used to minimize the likelihood or impact of potential problems. Risk management means dealing with a concern before it becomes a crisis. This improves the chance of project success and reduces the financial or other consequences of those risks that you can't avoid. Risks that lie outside the team's sphere of control should be directed to the appropriate level of management for attention.

Because requirements play such a central role in software projects, the prudent project manager will identify requirements-related risks early and control them aggressively. Typical requirements risks include misunderstanding the requirements, inadequate user involvement, uncertain or changing project scope and objectives, and continually changing requirements. Project managers can control requirements risks only through collaboration with customers and other stakeholders. Jointly documenting requirements risks and planning mitigation actions reinforces the customer-development partnership that was discussed in Chapter 2, "Requirements from the customer's perspective."

Simply knowing about the risks doesn't make them go away, so this chapter presents a brief tutorial on software risk management (Wiegers 2007). Later in the chapter, we also describe a number of risk factors that can raise their ugly heads during requirements engineering activities. Use this information to launch an attack on your requirements risks before they attack your project.

Fundamentals of software risk management

Projects face many kinds of risks besides those related to requirements. Dependence on an external entity, such as a subcontractor or another project that is providing components to be reused, is a common source of risk. Project management is fraught with risks from poor estimation, rejection of accurate estimates by managers, insufficient visibility into project status, and staff turnover. Technology risks threaten highly complex and leading-edge development projects. Lack of knowledge is another source of risk, such as with practitioners who have insufficient experience with the technologies being used or with the application domain. Transitioning to a new development method introduces a raft of new risks. And ever-changing, imposed government regulations can disrupt the best-laid project plans.

Scary! This is why all projects need to take risk management seriously. Risk management involves scanning the horizon for icebergs, rather than steaming full speed ahead with great confidence that your ship is unsinkable. As with other processes, scale your risk management activities to your project's size. Small projects can get by with a simple risk list, but formal risk management planning is a key element of a successful large-scale project.

Elements of risk management

Risk management involves the application of tools and procedures to contain project risk within acceptable limits. It provides a standard approach to identify and document risk factors, evaluate their potential severity, and propose strategies for mitigating them (Williams, Walker, and Dorofee 1997). Risk management includes the activities shown in Figure 32-1 (adapted from McConnell [1996]).

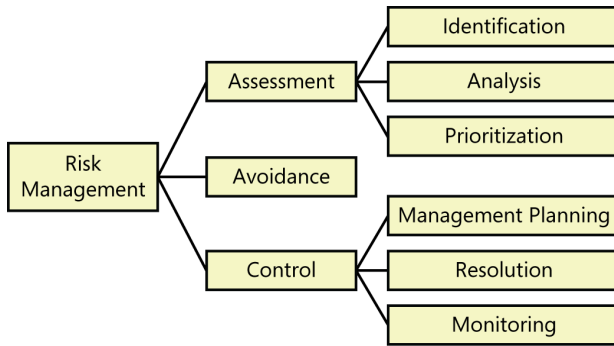


FIGURE 32-1 Elements of risk management.

Risk assessment is the process of examining a project to identify potential threats. Facilitate *risk identification* with lists of common risk factors such as those described in the “Requirements-related risks” section later in this chapter or with other public lists of typical risks (for example, Carr et al. 1993; McConnell 1996). During *risk analysis*, you’ll examine the potential consequences of specific risks to your project. *Risk prioritization* helps you focus on the most severe risks by assessing the potential *risk exposure* from each. Risk exposure is a function of both the probability of incurring a loss due to the risk and the potential magnitude of that loss.

Risk avoidance is one way to deal with a risk: don’t do the risky thing. You can avoid some risks by not undertaking certain projects, by relying on proven rather than cutting-edge technologies and development methods, or by excluding features that will be especially difficult to implement. Software development is intrinsically risky, though, so avoiding risk might also mean losing opportunities.

Most of the time you’ll have to perform *risk control* activities to manage the top-priority risks you identified. *Risk management planning* produces a plan for dealing with each significant risk, including mitigation approaches, contingency plans, owners, and timelines. Mitigation actions try either to prevent the risk from becoming a problem at all or to reduce the adverse impact if it does. The risks won’t control themselves, so *risk resolution* involves executing the plans for mitigating each risk. Finally, track your progress toward resolving each risk item through *risk monitoring*, which should become part of your routine project status tracking. Monitor how well your risk mitigation actions are working, look for new risks that have popped up, retire risks whose threat has passed, and update the priorities of your risk list periodically.

Documenting project risks

It’s not enough to simply recognize the risks that face your project. You need to manage them in a way that lets you communicate risk issues and status to stakeholders throughout the project’s duration. Figure 32-2 shows a template for documenting an individual risk statement. You might find it more convenient to store this information in tabular form, such as a spreadsheet, which makes it easy to sort the list of risks in various ways, or in a database. Keep the risk list separate from project plans so that it’s easy to update throughout the project’s duration.

ID: <sequence number>
Submitter: <individual who brought this risk to the team's attention>
Date Opened: <date the risk was identified>
Date Closed: <date the risk was closed out>
Risk Statement: <statement of the risk in the form "condition-consequence">
Scope of Impact: <project teams, business areas, and functional areas the risk could affect>
Probability: <the likelihood of the risk becoming a problem>
Impact: <numerical rating of the potential damage if the risk does become a problem>
Exposure: <probability multiplied by impact>
Risk Management Plan: <one or more approaches to control, avoid, minimize, or otherwise mitigate the risk>
Contingency Plan: <course of action to follow if the risk management plan is not effective>
Owner: <individual responsible for resolving the risk>
Date Due: <date by which the mitigation actions are to be implemented>

FIGURE 32-2 Risk item tracking template.

Use a *condition-consequence* format when you document risk statements. That is, state the risk condition that you are concerned about, followed by the potential adverse outcome—the consequence—from that condition. Often, people who suggest risks state only the condition (“the customers don’t agree on the product requirements”) or the consequence (“we can satisfy only one of our major customers”). Pull these statements together into the condition-consequence structure: “If the customers don’t agree on the product requirements, then we might be able to satisfy only one of our major customers.” One condition might lead to several consequences, and several conditions can result in the same consequence.

The template provides spaces to record the probability of a risk materializing into a problem, the negative impact on the project as a result of that problem, and the overall risk exposure. I like

to estimate the probability on a scale from 0.1 (highly unlikely) to 1.0 (certain to happen), and the impact on a relative scale of 1 (no problem) to 10 (big trouble). Even better, try to rate the potential impact in units of lost time or money. Multiply the probability by the impact to estimate the exposure from each risk.

Don't try to quantify risks too precisely. Your goal is to differentiate the most threatening risks from those you don't need to tackle immediately. You might find it easier simply to estimate both probability and impact as high, medium, or low. Those items that have at least one *high* rating demand your early attention.

Use the Risk Management Plan field to identify the actions you intend to take to control the risk. Some mitigation strategies work to reduce the risk probability, others to reduce the impact. Consider the cost of mitigation when planning. It doesn't make sense to spend \$20,000 to control a risk with a maximum estimated impact of only \$10,000 if it materialized into a problem. You might also devise contingency plans for the most severe risks to anticipate what actions to take if, despite your efforts, the risk does affect your project. Assign every risk that you plan to control to an individual owner, and set a target date for completing the mitigation actions. Long-term or complex risks might require a multistep mitigation strategy.

Figure 32-3 illustrates a risk that the Chemical Tracking System team leaders discussed at the beginning of this chapter. The team estimated the probability and impact on the basis of their previous experience. Until they evaluate other risks, they won't know how serious a risk exposure of 4.2 is—risk exposures are relative. The first two mitigation approaches reduce the probability of this risk becoming a problem by increasing user involvement in the requirements process. Prototyping reduces the potential impact by seeking early feedback on the user interface.

ID: 1		Submitter: Yuhong Li	
Date Opened: 8/22/13		Date Closed: (open)	
Risk Statement: If we have insufficient user involvement in requirements elicitation, then we might need to perform extensive user interface rework after beta testing.			
Scope of Impact: Could affect entire system, including any customizations done on integrated COTS components.			
Probability: 0.6		Impact: 7	Exposure: 4.2
Risk Management Plan: 1. Specify usability requirements during early elicitation activities. 2. Hold facilitated workshops with product champions to develop the requirements. 3. Develop a throwaway mock-up prototype of core functionality with input from product champions. 4. Have members of several user classes evaluate the prototype.			
Contingency Plan: Bring in an expert usability consultant to ensure that the UI conforms to established best practices for human-computer interface design.			
Owner: Reina Cabatana		Date Due: 2/13/14	

FIGURE 32-3 Sample risk item from the Chemical Tracking System.

Planning for risk management

A risk list is not the same as a risk management plan. For a small project, you can include your plans for controlling risks in the software project management plan. A large project should write a separate risk management plan that spells out the approaches it intends to take to identify, evaluate, document, and track risks. This plan should include the roles and responsibilities for the risk management activities. A risk management plan template is available with this book's companion content. Many projects appoint a project risk manager to be responsible for staying on top of the things that could go wrong. One company dubbed their risk manager "Eeyore," after the gloomy Winnie-the-Pooh character who constantly bemoaned how bad things could become.



Trap Don't assume that risks are under control just because you identified them and selected mitigation actions. Follow through on the risk management actions. Include enough time for risk management in the project schedule so that you don't waste your investment in risk planning. Include risk mitigation activities, risk status reporting, and updating the risk list in your project's task list.

Establish a rhythm of periodic risk monitoring. Keep the 10 or so risks that have the highest risk exposure visible, and track the effectiveness of your mitigation approaches regularly. When a mitigation action is completed, reevaluate the probability and impact for that risk item and then update the risk list and any other pending mitigation plans accordingly. A risk is not necessarily under control simply because the mitigation actions have been completed. You need to judge whether your mitigation approaches have reduced the exposure to an acceptable level or whether the opportunity for a specific risk to become a problem has passed.



Out of control

A project manager once asked me what to do if the same items remained on his top-five risk list week after week. This suggests that the mitigation actions for those risks aren't being implemented, that they aren't effective, or that there's no way for the team to control those risks. If your mitigation actions are effective, the exposure from risks that you are attempting to control will decrease. This lets other risks that were less threatening than the initial top five float up to the top of the risk list and engage your attention. Periodically reassess the probability of each risk materializing, and the potential loss if it does, to see whether your risk mitigation activities are getting the job done.

Requirements-related risks

The risk factors described on the following pages are organized by the five requirements engineering subdisciplines of elicitation, analysis, specification, validation, and management. Techniques are suggested that can reduce each risk's probability or impact. This list is just a starting point;

accumulate your own list of risk factors and mitigation strategies based on the lessons you learn from each project. Theron Leishman and David Cook (2002) describe additional risks related to software requirements. Be sure to write your risk statements in the condition-consequence format.

Requirements elicitation

Numerous factors can conspire to hamper your requirements elicitation efforts. Following are several areas of potential elicitation risk and suggestions for how to avoid them.

Product vision and project scope Scope creep is more likely if the stakeholders lack a shared understanding of what the product is supposed to be (and not be) and do. Early in the project, write a vision and scope document that contains your business requirements, and use it to guide decisions about new or modified requirements.

Time spent on requirements development Tight project schedules often pressure managers and customers into glossing over the requirements because they believe that if the developers don't start coding immediately, they won't finish on time. Record how much effort you actually spend on requirements development for each project so that you can judge whether it was sufficient and improve your planning for future projects. Agile development approaches allow construction to begin earlier than on projects following a waterfall life cycle.

Customer engagement Insufficient customer involvement during the project increases the chance of an expectation gap. Identify stakeholders, customers, and user classes early in the project. Determine who will serve as the literal voice of the user for each user class. Engage key stakeholders as product champions. Make sure product champions fulfill their commitments so you elicit the correct needs.

Completeness and correctness of requirements specifications Elicit user requirements that map to business requirements to ensure that the solution will deliver what the customers really need. Devise usage scenarios, write tests from the requirements, and have customers define their acceptance criteria. Create prototypes to make the requirements more meaningful for users and to elicit specific feedback from them. Enlist customer representatives to review the requirements and analysis models.

Requirements for innovative products It's easy to misgauge market response to products that are the first of their kind. Emphasize market research, build prototypes, and use focus groups to obtain early and frequent customer feedback about your innovative product visions.

Defining nonfunctional requirements Because of the natural emphasis on product functionality, it's easy to neglect nonfunctional requirements. Query customers about quality characteristics such as performance, usability, security, and reliability. Document these nonfunctional requirements and their acceptance criteria as precisely as you can.

Customer agreement on requirements If the diverse customers for your system don't agree on what you should build, someone will be unhappy with the result. Determine who the primary customers are, and use the product champion approach to get adequate customer representation

and involvement. Make sure you're relying on the right people for making decisions about requirements. Have appropriate stakeholder representatives review the requirements.

Unstated requirements Customers often hold implicit expectations that are neither communicated nor documented. Try to identify any assumptions the customers might be making. Use open-ended questions to encourage customers to share more of their thoughts, wishes, ideas, information, and concerns than you might otherwise hear. Asking customers what would make them reject the product might reveal some topics that have not yet been explored.

Existing product used as the requirements reference Requirements development might not be deemed important on next-generation or replacement projects. Developers are sometimes told to use the existing product as their source for requirements, with a list of changes and additions. Chapter 21, "Enhancement and replacement projects," suggested some ways to reverse-engineer requirements from an existing application.

Solutions presented as needs User-proposed solutions can mask the users' actual needs, lead to automating ineffective business processes, and overconstrain the developers' design options. The analyst must drill down to understand the intent—the real requirement—behind a solution the customer has presented.

Distrust between the business and the development team As you have seen throughout this book, effective requirements engineering demands close collaboration among various stakeholders, particularly customer communities (the business side for IT projects) and developers. If these parties do not feel that their counterparts are working in good faith toward a mutually beneficial outcome, conflicts can arise and requirements elicitation can be threatened.

Requirements analysis

It isn't prudent to just record whatever the customer tells you and dive into development. Requirements analysis poses its own threat areas, as described below.

Requirements prioritization Ensure that every functional requirement, feature, or user requirement is prioritized and allocated to a specific system release or iteration. Evaluate the priority of new requirements against the backlog of work remaining to be done, so that you can make appropriate trade-off decisions and iteration plans.

Technically difficult features Evaluate the feasibility of each requirement to identify those that might take longer than anticipated to implement. Use status tracking to watch for requirements that are falling behind their implementation schedule. Take corrective action as early as possible. Prototype the novel or risky requirements to select effective approaches.

Unfamiliar technologies, methods, languages, tools, or hardware Don't underestimate the learning curve of getting up to speed with new techniques that are needed to satisfy certain requirements. Identify those high-risk requirements early on, and work with the development team to allow sufficient time for false starts, learning, and experimentation.

Requirements specification

Requirements are all about communication. Just because requirements are communicated on paper or in writing doesn't mean they are actually understood.

Requirements understanding Different interpretations of the requirements by developers and customers lead to expectation gaps, in which the delivered product fails to satisfy customer needs. Peer review of requirements by developers, testers, and customers can mitigate this risk. Trained and experienced business analysts will acquire the right information and write high-quality specifications. Creating models and prototypes that represent the requirements from multiple perspectives can reveal fuzzy, ambiguous requirements.

Time pressure to proceed despite open issues It is a good idea to mark areas of the requirements that need further work with TBD (to be determined) or as issues, but it's risky to proceed with construction if these haven't been resolved. Record who is responsible for closing each open issue and the target date for resolution.

Ambiguous terminology Create a glossary to define business and technical terms that might be interpreted differently by different readers. Requirements reviews can help participants reach a common understanding of terms and concepts.

Design included in requirements Design elements that are included in the requirements place constraints on the options available to developers. Unnecessary constraints inhibit the creation of optimal designs. Review the requirements to make sure they emphasize what needs to be done to solve the business problem, rather than dictating the solution.

Requirements validation

Even if you've done a good job on requirements elicitation, it's important to confirm the quality and validity of the solution that the requirements specify. Validation offers the following pitfalls.

Unvalidated requirements The prospect of reviewing a lengthy requirements specification is daunting, as is the idea of writing tests very early in the development process. However, if you confirm the correctness and quality of each set of requirements before their implementation, you can avoid considerable expensive rework later. Include time and resources for these quality activities in the project plan. Gain commitment from your customer representatives to participate in requirements reviews. Perform incremental, informal reviews to find problems as early and cheaply as possible.

Inspection proficiency If inspection participants do not know how to inspect requirements effectively, they might miss serious defects. Train all team members who will participate in inspections of requirements documents. Invite an experienced inspector from your organization or an outside consultant to observe your early inspections to coach the participants.

Requirements management

Much of the requirements-related risk on a software project comes from how changes are handled. Those and other requirements management risks are mentioned below.

Changing requirements You can control rampant scope creep by using documented business requirements and scope definitions as the benchmark for approving changes. A collaborative elicitation process with extensive user involvement can cut requirements creep nearly in half (Jones 1996a). Detecting requirements errors early reduces the number of modifications requested later on. Design the system for easy modifiability, particularly when you are following an iterative life cycle.

Requirements change process Risks related to how requirements changes are handled include not having a defined change process, using ineffective change mechanisms, failing to incorporate valuable changes efficiently, and incorporating changes that bypass the process. A requirements change process that includes impact analysis, a change control board, and a tool to support the process is an important starting point. Clear communication of changes to the affected stakeholders is essential.

Unimplemented requirements Requirements tracing helps you avoid overlooking any requirements during design, construction, or testing.

Expanding project scope If requirements are poorly defined initially, further clarification can expand the scope of the project. Vaguely specified areas of the product will consume more effort than anticipated. The project resources that were allocated according to the initial incomplete requirements might be insufficient to implement the full scope of user needs. To mitigate this risk, plan on a phased or incremental delivery life cycle. Implement the top priority functionality in the early releases, and elaborate the system's capabilities in later iterations.

Risk management is your friend

A project manager can use risk management to raise the awareness of conditions that could cause the project to suffer. Consider the manager of a new project who's concerned about getting appropriate users involved in requirements elicitation. The astute manager will realize that this condition poses a risk and will document it in the risk list, estimating the probability and impact based on previous experience. If time passes and users still are not involved, the risk exposure for this item will increase, perhaps to the point where it compromises the project's success. I've been able to convince managers to postpone a project that could not engage sufficient user representatives by arguing that we shouldn't waste the company's money on a doomed project.



Periodic risk tracking keeps the project manager apprised of the threat from identified risks. Escalate risks that aren't adequately controlled to senior managers, who can either initiate corrective actions or make a conscious business decision to proceed despite the risks. Risk management helps you keep your eyes open and make informed decisions, even if you can't control or avoid every adversity your project might encounter.



Next steps

- Identify several requirements-related risks facing your current project. Don't identify known problems as risks, only things that haven't happened yet. Document the risks by using the template in Figure 32-2. Suggest at least one possible mitigation approach for each risk that you choose to control. Are there any risks that you are going to simply accept and hope they don't bite you?
- Hold a risk brainstorming session with key project stakeholders. Identify as many requirements-related risks as you can. Evaluate each for its probability of occurrence and relative impact, and multiply these together to calculate the risk exposure. Sort the risk list in descending order by risk exposure to identify your top five requirements-related risks. For each one, identify actions that can be taken to mitigate the risk. Assign each action to an individual to implement.
- Build your own list of potential requirements risks facing your organization. Start with the ones in this chapter, then augment the list based on actual project experiences. This rich risk list will help each future project manager identify his own risks early on.

Epilogue

Nothing is more important to a software project's success than understanding what problems it needs to solve. Requirements provide the foundation for that success. If the development team and its customers don't agree on the product's capabilities and characteristics, the most likely outcome is one of those unpleasant software surprises that we'd all prefer to avoid. If your current requirements practices aren't giving you the results you need, selectively and thoughtfully apply the techniques presented in this book that you think might help. Effective requirements engineering involves:

- Engaging customer representatives early and extensively.
- Developing requirements iteratively and incrementally.
- Representing the requirements in various ways to make sure everyone understands them.
- Assuring the requirements' completeness and correctness with all stakeholder groups.
- Finding the right supporting technology and practices to enable a shared view and ensure requirements integrity.
- Controlling the way that requirements changes are made.

Changing the way an organization works is difficult. It's hard to acknowledge that your current approaches aren't working as well as you'd like and to figure out what to try next. It's hard to find the time to learn about new techniques, develop improved processes, pilot and adjust them, and roll them out to the rest of the organization. And it can be difficult to convince the various stakeholders that change is needed. However, if you don't change the way your teams work, there's no reason to believe that the next project will go any better than the last one.

Success in software process improvement depends on:

- Focusing on a few clear pain points at a time.
- Setting clear goals and defining action plans for your improvement activities.
- Addressing the human and cultural factors associated with organizational change.
- Persuading everyone to view process improvement as a strategic investment in business success.

Keep these process improvement principles in mind as you define a road map to improved requirements engineering performance. Stay grounded in practical approaches that are appropriate for your organization and team. If you actively apply known good practices and rely on common sense, you can significantly improve how you handle your project's requirements, with all the advantages and benefits that brings. And remember that without excellent requirements, software is like a box of chocolates: you never know what you're going to get.

Current requirements practice self-assessment

This appendix contains 20 questions that you can use to calibrate your team's current requirements engineering practices and to identify areas to reinforce. You can download a copy of this assessment and a spreadsheet to help you analyze the responses from the companion content website for this book. More comprehensive assessments are available if you want to get a more precise understanding of what aspects of your current practices and documents would benefit most from improvement. Seilevel (2012) offers a thorough project assessment that can be adapted to evaluate your organization's requirements practices and deliverables.

To complete the quick assessment in this appendix, select the response for each question that most closely describes the way your team currently deals with that requirements issue. If you want to quantify the self-assessment, give yourself 0 points for each (a) response, 1 point for each (b), 3 points for each (c), and 5 points for each (d) response [except for question 16, where both (c) and (d) are worth 5 points]. The maximum possible score is 100 points. Generally speaking, the higher the score, the more mature—and likely more effective—your requirements practices are. Each question refers you to the chapter or chapters that address the topic of the question.

Instead of just trying to achieve a high score, use this self-assessment to spot opportunities to apply new practices that might benefit your organization. Some questions might not pertain to the kind of software your organization develops. Also, situations are different; not every project needs the most rigorous approaches. Recognize, though, that informal approaches to requirements increase the likelihood that your team will end up doing excessive rework. Most organizations will benefit from following the practices represented by the "c" and "d" responses.

The people you select to complete the assessment could influence the results. Watch out for respondents who, rather than describing what's really going on in the organization, might bias their responses based on politics, on what they wish was being done, or on what they think the "correct" answers should be. Asking multiple people to complete the self-assessment independently will help remove some of that bias and provide a more realistic representation of your current practices than asking just one person. Multiple responders might also reveal different understandings of how certain practices are being performed at present. You can use the spreadsheet tool provided on the companion content website to accumulate multiple sets of responses and view the distribution.

- 1.** How are the project's business requirements defined, communicated, and used? [Chapter 5]
 - a.** We sometimes write a high-level product description early on, but we don't refer back to it.
 - b.** The person who conceives the product knows the business requirements and discusses them verbally with the development team.
 - c.** We record business requirements in a vision and scope, project charter, or similar document according to a standard template. All project stakeholders have access to this document.
 - d.** We actively use the documented business requirements on our project, evaluating proposed product features and requirement changes to see whether they lie within the documented scope, and adjusting scope as needed based on business objectives.
- 2.** How are the user communities for the product identified and characterized? [Chapter 6]
 - a.** The developers think they know who our users will be.
 - b.** Marketing or the project sponsor believes that they know who the users are.
 - c.** Target user groups or market segments are identified by management or marketing from some combination of market research, our existing user base, and input from other stakeholders.
 - d.** The project stakeholders identify distinct user classes, whose characteristics are summarized in the software requirements specification.
- 3.** How do you elicit customer input on the requirements? [Chapter 7]
 - a.** The developers are confident that they already know what to build.
 - b.** Typical users are surveyed with questionnaires or interviewed in focus groups.
 - c.** We meet with people, sometimes one on one and sometimes in groups, and they tell us what they want.
 - d.** A variety of elicitation techniques are used, including interviews and workshops with user class representatives, document analysis, and system interface analysis.
- 4.** How well trained and how experienced are your business analysts? [Chapter 4]
 - a.** They are developers or former users who have little experience and no specific training in software requirements engineering.
 - b.** Developers, experienced users, or project managers who have had some previous exposure to requirements engineering perform the BA role.
 - c.** The BAs have had several days of training and considerable experience in collaborating with users.

- d.** We have professional business analysts or requirements engineers who are trained and proficient in interviewing techniques, the facilitation of group sessions, technical writing, and modeling. They understand both the application domain and the software development process.
- 5.** How are the high-level system requirements allocated to the software portions of the product? [Chapters 19 and 26]
 - a.** Software is expected to overcome any shortcomings in the hardware.
 - b.** Software and hardware engineers discuss which subsystems should perform which functions.
 - c.** A system engineer or an architect analyzes the system requirements and decides which ones will be implemented in each software subsystem.
 - d.** Knowledgeable team members collaborate to allocate portions of the system requirements to software subsystems and components and to trace them into specific software requirements. Component interfaces are explicitly defined and documented.
- 6.** To what extent are requirements reused on your projects? [Chapter 18]
 - a.** We do not reuse requirements.
 - b.** A business analyst who is familiar with previous projects sometimes knows of requirements that can be reused on a new project, so she copies and pastes the requirements into the new specification.
 - c.** A business analyst can search through the previous projects stored in our requirements management tool for requirements that are relevant to his new project. He can reuse specific versions of those requirements by using the functions built into the tool.
 - d.** We have established a repository of potentially reusable requirements, which have been adapted and improved from previous projects. BAs routinely check this repository for requirements that might be usable on their current projects. We use trace links to pull in child requirements, dependent requirements, design elements, and tests when possible when we are reusing a requirement.
- 7.** What approaches are used when working with stakeholders to identify the specific software requirements? [Chapters 7, 8, 12, and 13]
 - a.** We begin with a general understanding, write some code, show the software to some users, and modify the code until they're happy.
 - b.** Management or marketing provides a product concept, and the developers write the requirements. Customer stakeholders tell the development team if they've missed anything.

- c. Marketing or customer representatives tell the development team what features and functions the product should contain. Sometimes marketing tells the development team when the product direction changes.
 - d. We hold structured requirements elicitation interviews or workshops with representatives from the different user classes for the product. We employ use cases or user stories to understand the users' goals, and we create analysis models to help ensure we identify all the functional requirements. We flesh out the requirements incrementally and iteratively, giving the customers numerous opportunities to improve them.
- 8. How are the software requirements documented? [Chapters 10, 11, 12, and 30]
 - a. We piece together oral history, email and voice mail messages, interview notes, and meeting notes.
 - b. We write unstructured narrative textual documents, or we create simple requirements lists, or we draw some diagrams.
 - c. We write requirements in structured natural language according to a standard template. Sometimes we augment these requirements with visual analysis models that use standard notations.
 - d. We create requirements and visual analysis models and store them all in a requirements management tool. Several attributes are stored along with each requirement.
- 9. How are nonfunctional requirements, such as software quality attributes, elicited and documented? [Chapter 14]
 - a. What are "software quality attributes"?
 - b. We do beta testing to get feedback about how the users like the product.
 - c. We document certain attributes, such as performance, usability, and security requirements.
 - d. We work with customers to identify the important quality attributes for each product, which we then document in a precise and verifiable way.
- 10. How are the individual functional requirements labeled? [Chapter 10]
 - a. We write paragraphs of narrative text or short user stories; specific requirements are not explicitly identified.
 - b. We use bulleted or numbered lists.
 - c. We use a hierarchical numbering scheme, such as "3.1.2.4."
 - d. Each discrete requirement has a unique, meaningful label that is not disrupted when other requirements are added, moved, or deleted.

- 11.** How are priorities for the requirements established? [Chapter 16]
- a.** All of the requirements are important, so we don't need to prioritize them.
 - b.** The customers tell us which requirements are most important to them. If the customers don't tell us or don't agree, the developers decide.
 - c.** Each requirement is labeled as high, medium, or low priority by customer consensus.
 - d.** To help us make priority decisions, we use an analytical process to rate the value, the cost, and the technical risk of each requirement, or we use a similar structured prioritization technique.
- 12.** What techniques are used to prepare a partial solution and verify a mutual understanding of the problem? [Chapter 15]
- a.** We just build the system and then fix it if we need to.
 - b.** We build some simple prototypes and ask users for feedback. Sometimes we're pressured to deliver prototype code.
 - c.** We create prototypes for both user interface mock-ups and technical proofs of concept when appropriate.
 - d.** Our project plans include tasks to create electronic or paper throwaway prototypes to help us refine the requirements. Sometimes we build evolutionary prototypes. We use evaluation scripts to obtain customer feedback on our prototypes.
- 13.** How are the requirements validated? [Chapter 17]
- a.** We think our requirements are pretty good when we first write them.
 - b.** We pass the specified requirements around to people to get their feedback.
 - c.** The BA and some stakeholders hold informal reviews when they have time.
 - d.** We inspect our requirements documents and models, with participants that include customers, developers, and testers. We write tests against the requirements and use them to validate the requirements and models.
- 14.** How are different versions of the requirements documents distinguished? [Chapters 27 and 30]
- a.** The document shows the auto-generated date that the document was printed.
 - b.** We use a sequence number—like 1.0, 1.1, and so on—for each document version.
 - c.** We have a manual identification scheme that distinguishes draft versions from baselined versions and major revisions from minor revisions.
 - d.** The requirements documents are stored under version control in a document management system, or requirements are stored in a requirements management tool that maintains a revision history for each requirement.

- 15.** How are software requirements traced back to their origin? [Chapter 29]
- a.** They aren't.
 - b.** We know where many of the requirements came from but don't document the knowledge.
 - c.** Each requirement has an identified origin.
 - d.** We have full two-way tracing between business requirements, system requirements, user requirements, functional requirements, and nonfunctional requirements.
- 16.** How are requirements used as the basis for developing project plans? [Chapter 19]
- a.** The delivery date is set before we begin requirements development. We can't change either the project schedule or the scope. Sometimes we go through a rapid descoping phase to drop features just before the delivery date.
 - b.** The first iteration of the project plan addresses the schedule needed to gather requirements. The rest of the project plan is developed after we have a preliminary understanding of the requirements. We can't change the plan much thereafter, however.
 - c.** We start with just enough information about requirements to prioritize them, then estimate the effort needed to implement the top-priority requirements. We develop our requirements and our software incrementally, planning the requirements for each iteration based on their priority and size. If we need to accommodate more requirements than our plan allowed, we add more iterations.
 - d.** We base the schedules and plans on the estimated effort needed to implement the required functionality, starting with the highest-priority requirements. These plans are updated as the requirements change. If we must drop features or adjust resources to meet schedule commitments, we do so as early as possible. We plan to deliver multiple releases to accommodate requirements changes and growth. [Note: (c) and (d) are equally good responses for this question.]
- 17.** How are the requirements used as a basis for design? [Chapter 19]
- a.** When we have written requirements, we refer to them during development.
 - b.** The requirements documents describe the solution we intend to implement.
 - c.** Each functional requirement is traced to a design element.
 - d.** Developers inspect the requirements to make sure they can be used as the basis for design. We have full two-way traceability between individual functional requirements and design elements.

- 18.** How are the requirements used as the basis for testing? [Chapter 19]
- a.** The testers test the software based on how they think it should function.
 - b.** The testers test what the developers said they implemented.
 - c.** We write system tests against the user requirements and functional requirements.
 - d.** Testers inspect the requirements to make sure they are verifiable and to begin their test planning. We trace system tests to specific functional requirements. System testing progress is measured in part by requirements coverage.
- 19.** How is a software requirements baseline defined and managed for each project? [Chapters 2 and 27]
- a.** We don't have to think about baselines because we are on an agile project.
 - b.** The customers and managers sign off on the requirements, but the development team still gets a lot of changes and complaints.
 - c.** We define an initial requirements baseline, but we don't always keep it current as changes are made over time.
 - d.** The requirements are stored in a requirements management tool when an initial baseline is defined. The requirements repository is updated as requirements changes are approved. We maintain a change history for each requirement after it's baselined. On an agile project, the team agrees on a requirements baseline for each iteration.
- 20.** How are changes to the requirements managed? [Chapter 28]
- a.** The requirements change whenever someone has a new idea or realizes that he forgot something.
 - b.** We discourage change by freezing the requirements after the requirements phase is complete, but informal change agreements are still made.
 - c.** We use a defined format for submitting change requests and a central submission point. The project manager decides which changes to incorporate.
 - d.** Changes are made according to our documented change control process. We use a tool to collect, store, and communicate change requests. The impact of each change is evaluated before the change control board decides whether to approve it.

